Krishna Dalvi

D20C/10

Blockchain : Experiment - 2

Aim:

Create a Blockchain using Python.

Theory:

Blockchain is a distributed and immutable ledger that records data in a sequence of linked blocks. Each block contains a timestamp, proof (from Proof-of-Work), and the hash of the previous block, ensuring integrity and security.

In this program:

- **Block Creation** – A block is generated using create_block() with a proof and previous hash.
- **Proof-of-Work (PoW)** – Implemented to mine a block by solving a cryptographic puzzle (finding a hash with leading zeros).
- **Hashing** – SHA-256 algorithm ensures data immutability.
- **Validation** – is_chain_valid() checks that every block correctly references the previous hash and satisfies PoW conditions.
- **Flask Web API** – Routes /mine_block, /get_chain, and /is_valid allow mining, fetching the chain, and verifying blockchain integrity through a browser or API client.

This program runs on a local server and simulates mining a blockchain without a network of nodes, making it an ideal introductory model to understand blockchain basics.

Code:

```
import datetime import
hashlib import json
from flask import Flask, jsonify class
Blockchain:
  def _init_(self):
    self.chain = [] self.transactions = []
    self.create_block(proof=1,
previous_hash='0')

  def create_block(self, proof, previous_hash,
transactions=None):
    if transactions is None: transactions
      = []
    block = {
      'index': len(self.chain) + 1,
      'timestamp':
str(datetime.datetime.now()),
      'proof': proof,
      'previous_hash': previous_hash,
      'transactions': transactions

  def get_previous_block(self):
    return self.chain[-1]

  def create_Transactions(self):
    if
      self.transa
      ctions:
      previous_
      block =
self.get_previous_block()
      previous_proof =
      previous_block['proof'] proof =
self.proof_of_work(previous_proof)
      previous_hash =
self.hash(previous_bloc
      k)
      transactions_to_
      mine =
self.transactions.copy()
      self.transactions =
      [] block =
      self.create_block(
        proof,
        previous_has
        h,
        transactions_to_mine
      )
      retur
```

```
    n block
    return
      }
      self.chain.append(block) return block
      def proof_of_work(self,
        previous_proof): new_proof = 1
        check_proof = False
        while check_proof is False:
        hash_operation = hashlib.sha256(
          str(new_proof**2
-
previous_proof**2).encode
()
        ).hexdigest()
        if hash_operation[:3] == '000':
          check_proof = True
        else:
          new_proof += 1
      return new_proof

   def hash(self, block):
      encoded_block = json.dumps(
        block,
        sort_keys=True
      ).encode()
      return
hashlib.sha256(encoded_block).hexdigest()

   def is_chain_valid(self, chain):
      previous_block = chain[0]
      block_index = 1
      while block_index < len(chain):
        block = chain[block_index]
        if block['previous_hash'] !=
self.hash(previous_block):
          return False
        previous_proof =
        previous_block['proof'] proof =
        block['proof']
        hash_operation =
          hashlib.sha256( str(proof**2 -
previous_proof**2).encode()
        ).hexdigest()
        if hash_operation[:3] != '000':
          return False
        previous_block = block
        block_index += 1
      return True

app = Flask(_name_)
blockchain =
Blockchain()
None

   proof =
blockchain.proof_of_work(previous_pro
of)
   previous_hash =
blockchain.hash(previo
us_block)
   block =
     blockchain.create_
     block( proof,
     previous_hash,
     blockchain.transactio
     ns
   )
   blockchain.trans
   actions = []
   response = {
     'message': 'Congratulations, you
just mined a block!',
     'index': block['index'],
     'timestamp':
     block['timestamp'],
     'proof': block['proof'],
     'previous_hash':
     block['previous_hash'],
     'transactions': block['transactions']
   }
   return jsonify(response), 200

@app.route('/get_chain',
methods=['GET']) def
get_chain():
   response = {
     'chain':
     blockchain.chain,
     'length':
     len(blockchain.chain)
   }
   return jsonify(response), 200

@app.route('/is_valid',
methods=['GET']) def is_valid():
   valid =
     blockchain.is_chain
     _valid(
     blockchain.chain
   )
   if valid:
     response = {
       'message': 'All good. The Blockchain is
vali
```

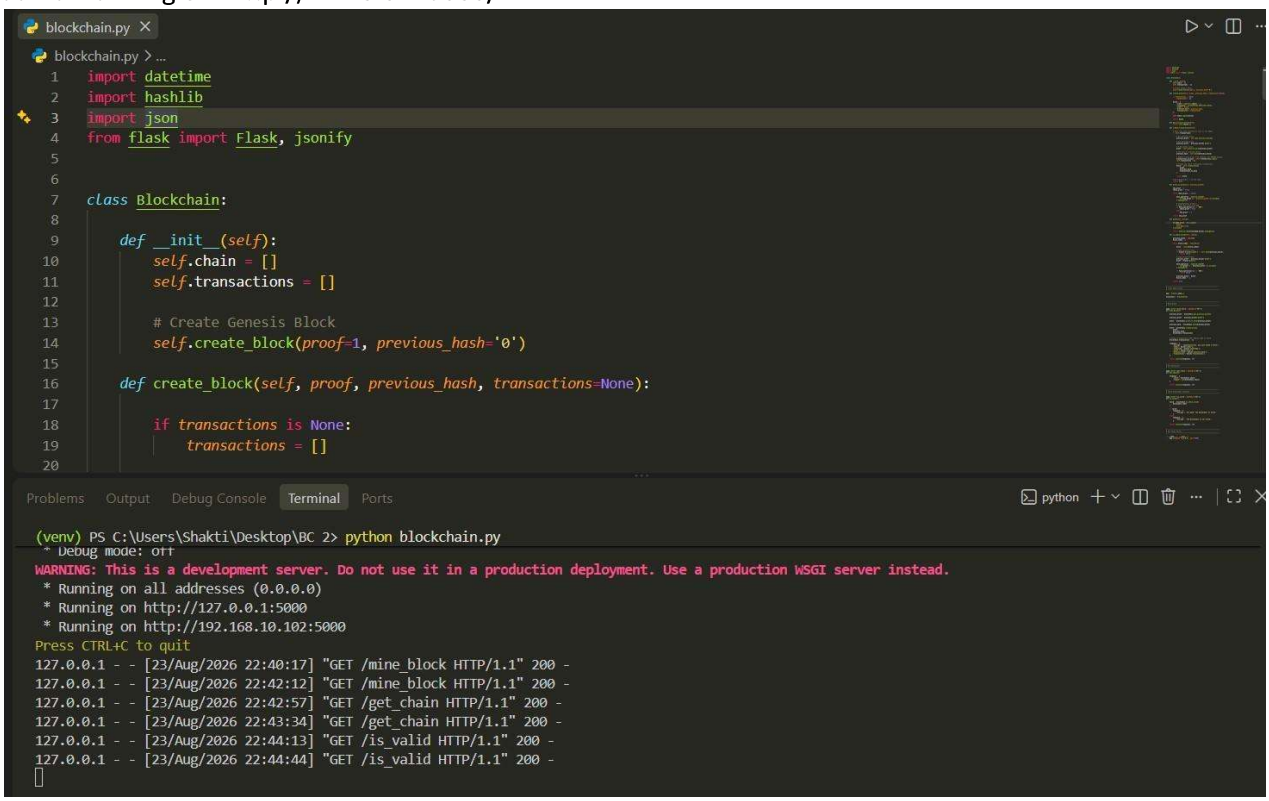
```
d.'
  }
else
            @app.route('/mine_block', methods=['GET']) def
            mine_block():
               previous_block = blockchain.get_previous_block()
               previous_proof = previous_block['proof']
```

```
:
                  response = {
                     'message': 'The Blockchain is not valid.'
                  }
               return jsonify(response), 200

            if __name____== '__main__':
               app.run(host='0.0.0.0',
               port=5000)
```

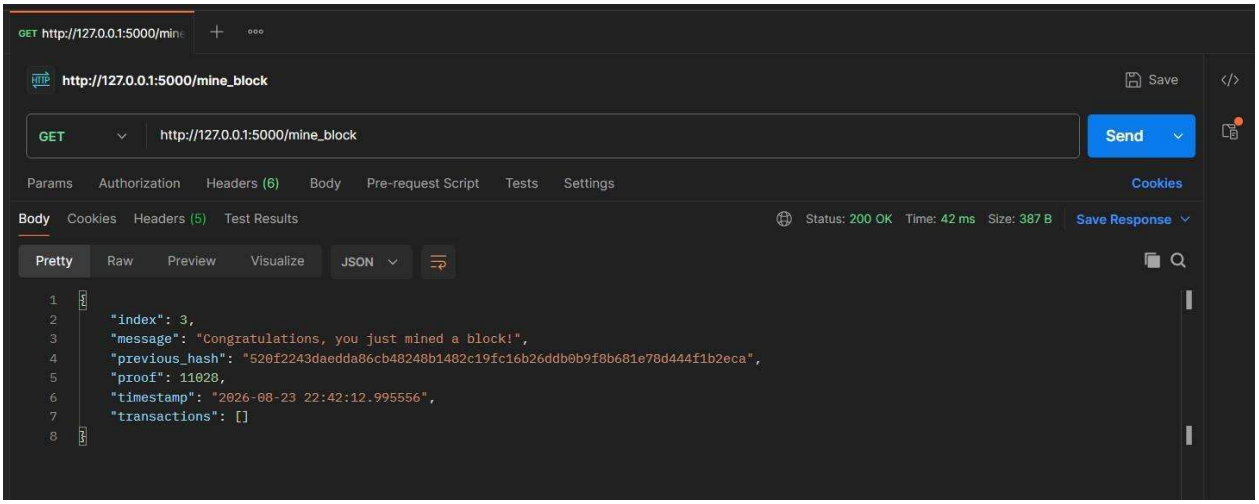
Output:

Server running on: http://127.0.0.1:5000/

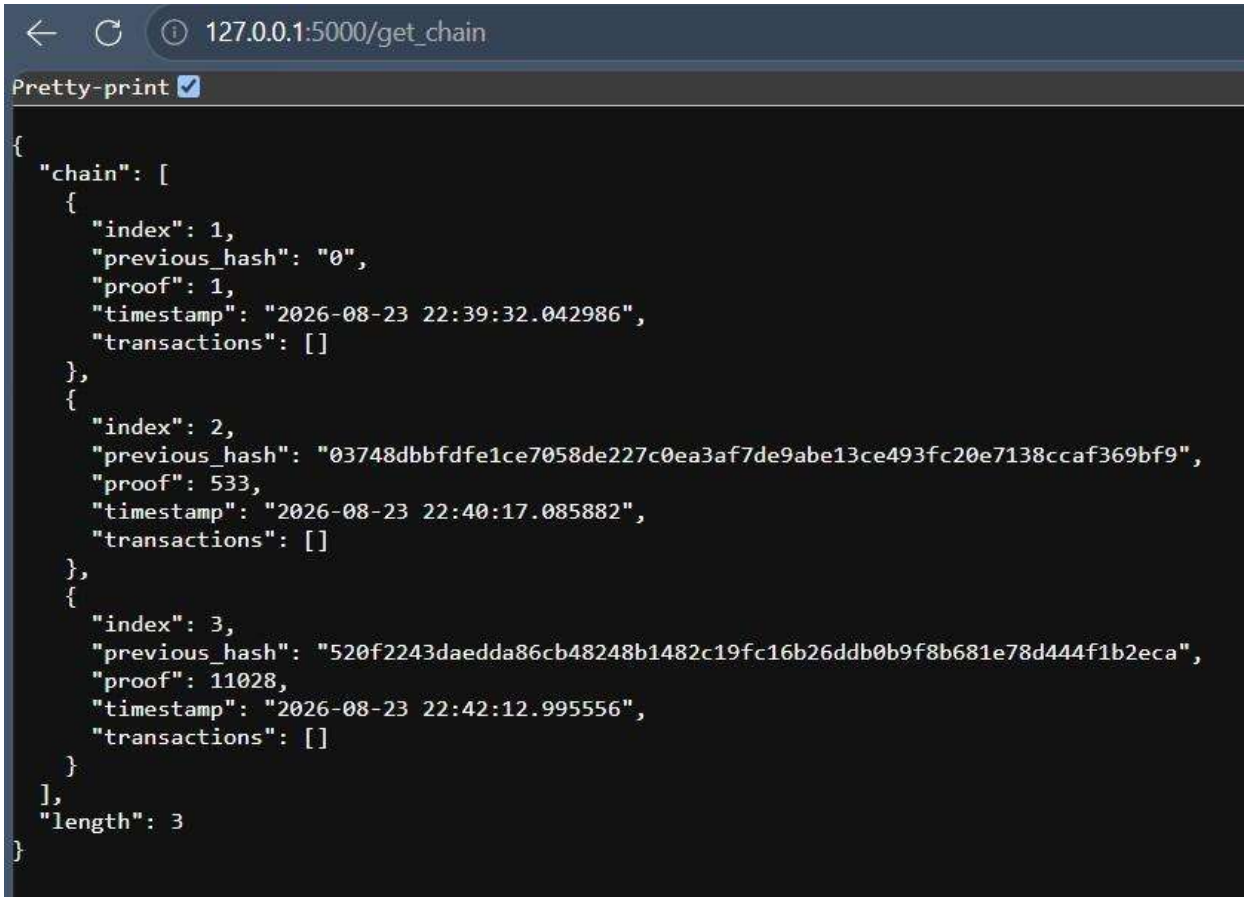


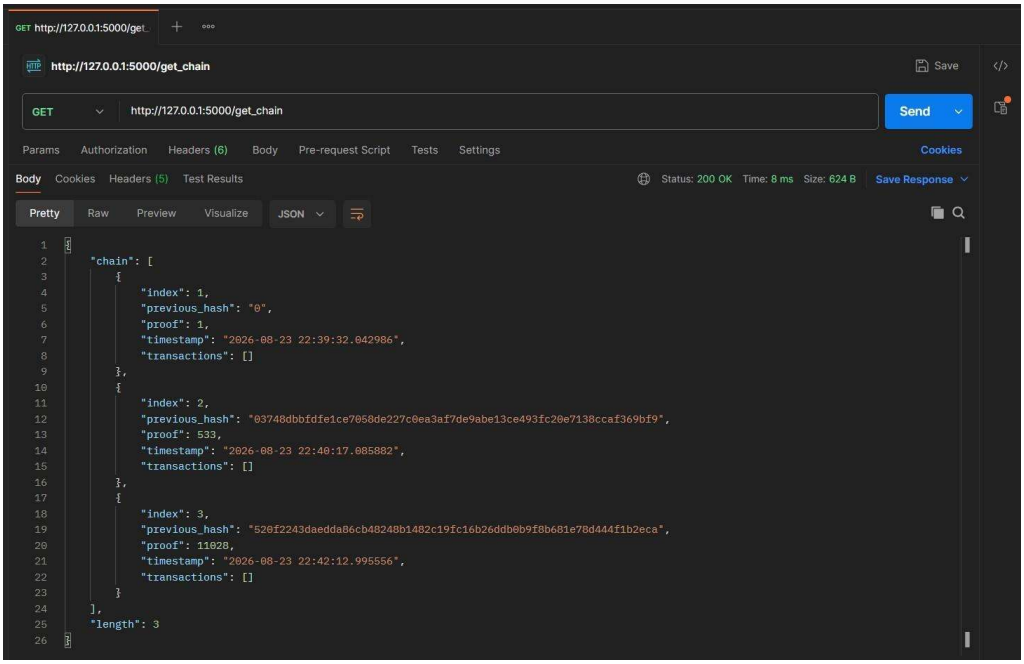
Output on http://127.0.0.1:5000/mine_block



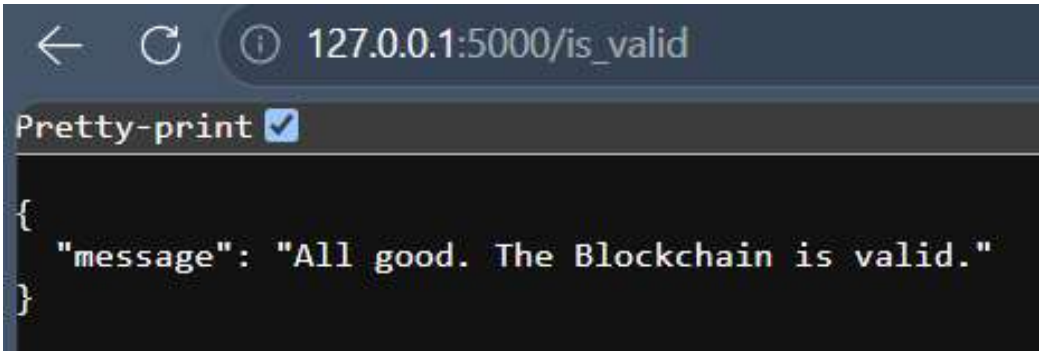


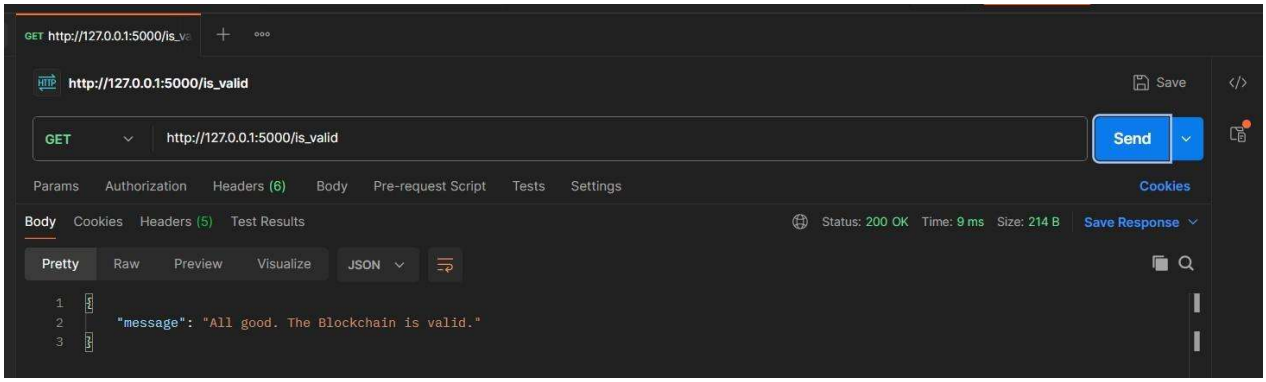
Output on http://127.0.0.1:5000/get_chain





Output on http://127.0.0.1:5000/is_valid





Conclusion:

We successfully created a basic blockchain using Python and Flask that supports block mining, chain retrieval, and validity checks. Proof-of-Work ensures security by making block creation computationally intensive, while cryptographic hashing guarantees immutability. The implementation demonstrates core blockchain principles—decentralization, immutability, and consensus—in a simplified environment, providing a strong foundation for building more advanced blockchain systems.